Hibernate & Spring Data JPA

Comprehensive Interview Q&A Guide

Complete reference with answers, code examples, and best practices

★ = Frequently Asked in Interviews

Table of Contents

Section 1: Hibernate Core

This section covers Hibernate ORM fundamentals, session management, entity mapping, caching, querying, and performance optimization. Questions marked with ★ are most frequently asked in interviews.

Part A: Core Concepts & Session Management

★ Q1: What is Hibernate and what problems does it solve over JDBC? [FREQUENTLY ASKED]

Hibernate is an Object-Relational Mapping (ORM) framework for Java that maps Java objects to relational database tables. It implements the JPA specification.

Problems solved over plain JDBC:

- Eliminates boilerplate: No manual ResultSet mapping, connection management, or PreparedStatement boilerplate
- Auto SQL generation: INSERT, UPDATE, DELETE, SELECT generated automatically from entity state
- Object-oriented querying: Use HQL/JPQL — work with objects and fields, not tables and columns
- Built-in caching: First-level (Session) and second-level (application-wide) caching
- Lazy loading: Related entities loaded only when accessed, reducing unnecessary queries
- Database portability: Switch databases by changing the Dialect — no query rewrites
- Transaction management: Integrated transaction abstraction over JDBC and JTA
- Schema generation: Auto-create/update/validate schema from entity definitions

Best Practice: Use Hibernate for complex object graphs. For performance-critical queries, combine with native SQL or Spring JDBC Template.

★ Q2: What is the difference between Hibernate and JPA? [FREQUENTLY ASKED]

JPA (Java Persistence API) is a specification — it defines interfaces, annotations, and rules. Hibernate is an implementation of JPA.

Aspect	JPA	Hibernate
Type	Specification (javax/jakarta.persistence)	Implementation (org.hibernate)
API	EntityManager, EntityManagerFactory	Session, SessionFactory
Portability	Swap to EclipseLink, OpenJPA, etc.	Hibernate-specific extras: @NaturalId, Envers
Extras	Standard only	Beyond-spec features available

You can code against JPA interfaces for portability, but Hibernate-specific features require the Hibernate API directly.

★ Q3: Explain the Hibernate architecture — SessionFactory, Session, Transaction. [FREQUENTLY ASKED]

- **SessionFactory: Thread-safe, heavyweight, application-scoped. Created once at startup. Holds connection pool, mapping metadata, and second-level cache. Treat as a singleton.**
- **Session: Lightweight, short-lived, NOT thread-safe. Represents one unit of work. Holds the first-level cache. Wraps a JDBC connection from the pool.**
- **Transaction: Abstracts JDBC or JTA transactions. Always begin/commit/rollback explicitly.**

Typical pattern:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
try { session.save(entity); tx.commit(); }
catch (Exception e) { tx.rollback(); }
finally { session.close(); }
```

★ Q4: What is the difference between Session.get() and Session.load()? [FREQUENTLY ASKED]

Feature	get()	load()
DB hit	Immediate SELECT fired	Returns proxy; SQL fires on first non-ID access
Not found	Returns null	Throws ObjectNotFoundException at access time
Use when	Unsure if entity exists	Setting FK where existence is guaranteed

```
// load() — no DB hit; just creates proxy for FK assignment
order.setCustomer(session.load(Customer.class, customerId));
```

Best Practice: Prefer get() in business logic. Use load() only to set FK references where you already know the entity exists.

★ Q5: What are the different states of a Hibernate entity — transient, persistent, detached, removed? [FREQUENTLY ASKED]

- Transient: Created with new — no Session, no DB row. GC'd if no references.
- Persistent: Linked to an open Session with a DB row. Changes auto-tracked via dirty checking.
- Detached: Session closed. Has an ID but changes not tracked. Re-attach via merge().
- Removed: Marked for deletion. Deleted from DB on flush/commit.

```
new Entity()            --> Transient
session.save(entity)    --> Persistent
session.close()         --> Detached
session.merge(entity)   --> Persistent (re-attached)
session.delete(entity)  --> Removed
```

★ Q6: What is the first-level cache in Hibernate and how does it work? [FREQUENTLY ASKED]

- Scope: Per Session — mandatory, cannot be disabled
- Stores entities by type + primary key
- Within the same session, repeated queries for the same ID return the cached object — no extra DB hit
- Dirty checking: Modified entities are tracked here and flushed together on commit
- Eviction: session.evict(entity) removes one; session.clear() removes all

```
User u1 = session.get(User.class, 1L); // DB query fires
User u2 = session.get(User.class, 1L); // from L1 cache, no DB hit
assert u1 == u2; // true — same Java object
```

Best Practice: Keep Sessions short-lived. Long-running Sessions accumulate all loaded entities in memory.

★ Q7: What is the second-level cache in Hibernate? How is it configured? [FREQUENTLY ASKED]

- Optional, application-scoped cache shared across Sessions (SessionFactory-level)
- Stores entity data (not Java objects) in per-entity regions
- Requires explicit opt-in via @Cache or @Cacheable on entity
- Common providers: EhCache, Redis (Redisson), Infinispan, Hazelcast

```
# application.properties
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=\
    org.hibernate.cache.ehcache.EhCacheRegionFactory
```

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product { ... }
```

Concurrency strategies: READ_ONLY (immutable), READ_WRITE (most common), NONSTRICT_READ_WRITE (slight staleness risk), TRANSACTIONAL (XA).

★ Q8: What is dirty checking in Hibernate and how does it work? [FREQUENTLY ASKED]

Dirty checking automatically detects changes to persistent entities and generates UPDATE SQL without explicit save calls.

- On flush/commit, Hibernate compares current entity state to snapshot taken at load time
- If any field differs, it generates UPDATE SQL for that entity
- Use @DynamicUpdate to generate SQL only for changed columns (helps wide tables)

```
User user = session.get(User.class, 1L); // snapshot saved
user.setEmail('new@email.com');          // modify
// NO session.update() call needed!
tx.commit(); // Hibernate detects change, auto-generates UPDATE
```

Best Practice: Avoid calling session.update() on already-persistent entities — dirty checking handles it automatically.

★ Q9: What is the N+1 problem in Hibernate and how do you solve it with JOIN FETCH? [FREQUENTLY ASKED]

N+1 occurs when Hibernate fires 1 query for a list, then N additional queries to load lazy associations for each item.

```
// N+1 problem:
List<Dept> depts = session.createQuery('FROM Department').list();
depts.forEach(d -> d.getEmployees().size()); // N extra queries!
```

Solutions:

- JOIN FETCH in HQL: FROM Department d JOIN FETCH d.employees
- @EntityGraph on repository method: @EntityGraph(attributePaths = {"employees"})
- @BatchSize(size=20) on collection — loads in batches instead of 1-by-1
- @Fetch(FetchMode.SUBSELECT) — loads all collections with one subquery

Best Practice: Always use JOIN FETCH or @EntityGraph for collections in list operations. Monitor SQL logs in dev to catch N+1 early.

★ Q10: What is a lazy initialization exception and how do you prevent it? [FREQUENTLY ASKED]

LazyInitializationException is thrown when accessing a lazy collection or proxy after the Session is closed.

```
User user = userRepository.findById(1L).get();
// Session closed after repository returns
user.getOrders().size(); // LazyInitializationException!
```

Prevention:

- Use JOIN FETCH or @EntityGraph to eagerly load needed associations in the same query
- Add @Transactional on the service method to keep Session open
- Use DTOs/projections — no lazy loading involved
- Call Hibernate.initialize(entity.getOrders()) inside an active session

Best Practice: The root fix is to load everything you need within the transaction. Design queries to return complete data.

★ Q11: How does Hibernate implement optimistic locking vs pessimistic locking? [FREQUENTLY ASKED]

Optimistic Locking:

- Assumes conflicts are rare — no DB locks held during the transaction
- Uses @Version field (integer or timestamp) — Hibernate checks version on UPDATE
- Throws StaleObjectStateException if version mismatches (someone else updated first)

```
@Entity public class Account {
    @Version private Long version; // auto-managed
}
```

Pessimistic Locking:

- Acquires real DB locks immediately — prevents concurrent reads/writes
- PESSIMISTIC_READ: Shared lock; PESSIMISTIC_WRITE: SELECT FOR UPDATE

```
session.get(Account.class, id, LockMode.PESSIMISTIC_WRITE);
```

- Optimistic: Low-contention, high-read web apps (most cases)
- Pessimistic: High-contention financial/critical operations

Part B: Mappings, Inheritance & Performance

★ Q12: What are the different DDL auto strategies? [FREQUENTLY ASKED]

Strategy	Behavior	Environment
none	No schema action	Production
validate	Validates schema vs entities; fails on mismatch	Production
update	Adds missing columns/tables; never drops	Dev only
create	Drops + recreates schema on startup	Dev / Testing
create-drop	Creates on start, drops on stop	Integration tests

Best Practice: Always use validate or none in production. Manage schema changes with Flyway or Liquibase.

★ Q13: What are the different inheritance mapping strategies in Hibernate? [FREQUENTLY ASKED]

- SINGLE_TABLE: All subclasses in one table with a discriminator column. No JOINs — best performance. Subclass-specific columns are nullable.
- TABLE_PER_CLASS: Each concrete class gets its own table with all inherited fields. UNION queries needed for polymorphic queries.
- JOINED: Each class has a table with only its own fields. Parent PK is FK in child. Best normalization but slowest due to JOINs.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Vehicle { @Id Long id; String brand; }
@Entity public class Car extends Vehicle { int doors; }
```

★ Q14: What are the trade-offs between SINGLE_TABLE, TABLE_PER_CLASS, and JOINED inheritance? [FREQUENTLY ASKED]

Criterion	SINGLE_TABLE	TABLE_PER_CLASS	JOINED
Performance	Best (no JOIN)	Medium (UNION ALL)	Slowest (JOINs)

Polymorphic query	Single scan	UNION ALL required	JOIN per subtype
Data integrity	NULLs for subtype cols	Enforced per table	Best (normalized)
Storage	Wasteful nulls	Duplicates columns	Normalized
Best use	Simple hierarchies	Rarely queried base	Complex domains

Best Practice: SINGLE_TABLE is the JPA default and usually best performer. Use JOINED when subtype fields must be NOT NULL or you need normalization.

★ Q15: What are HQL and Criteria API? When would you use one over the other?
[FREQUENTLY ASKED]

HQL (Hibernate Query Language):

- Object-oriented, SQL-like language using entity names and fields
- String-based — parsed at runtime; readable for complex static queries

```
session.createQuery('FROM User u WHERE u.age > :age', User.class)
       .setParameter('age', 18).list();
```

Criteria API:

- Type-safe, programmatic — compile-time field name validation
- Best for dynamic queries where conditions vary at runtime

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<User> cq = cb.createQuery(User.class);
Root<User> root = cq.from(User.class);
cq.where(cb.gt(root.get('age'), 18));
```

★ Q16: What are named queries in Hibernate and what are their advantages?
[FREQUENTLY ASKED]

Named queries are pre-defined HQL/SQL queries associated with an entity and validated at startup.

```
@Entity
@NamedQuery(name='User.findByEmail',
  query='FROM User u WHERE u.email = :email')
public class User { ... }
```

- Validated at startup — syntax errors caught before runtime
- Parsed and compiled once — reused efficiently on each call
- Centralized query definition close to the entity
- Overridable in orm.xml without touching source code

★ Q17: What is a Hibernate proxy object and how does lazy loading use it?
[FREQUENTLY ASKED]

A Hibernate proxy is a runtime-generated subclass (via Byte Buddy/CGLIB) that acts as a placeholder for the real entity.

- session.load() returns a proxy with only the ID set — no DB query yet
- Proxy intercepts the first non-ID method call and fires SELECT to load real data
- Enables lazy loading: DB is hit only when data is actually needed

```
User proxy = session.load(User.class, 1L); // No SQL
Long id = proxy.getId();      // No SQL — ID already set
String name = proxy.getName(); // SELECT fires HERE
```

- Pitfall: instanceof may behave unexpectedly — use Hibernate.getClass(entity)
- Use Hibernate.initialize(proxy) to force-load inside an active session

★ Q18: How do you map composite primary keys using @EmbeddedId and @IdClass? [FREQUENTLY ASKED]

@EmbeddedId approach:

```
@Embeddable
public class OrderItemId implements Serializable {
    Long orderId; Long productId;
}
@Entity public class OrderItem {
    @EmbeddedId private OrderItemId id;
}
```

@IdClass approach:

```
@Entity @IdClass(OrderItemId.class)
public class OrderItem {
    @Id Long orderId; @Id Long productId;
}
```

Key requirements for the ID class: implement Serializable, override equals() and hashCode().

★ Q19: What are some common Hibernate performance pitfalls and how do you avoid them? [FREQUENTLY ASKED]

- N+1 queries: Use JOIN FETCH or @EntityGraph for collections
- Eager loading everything: Keep LAZY by default, load eagerly per-query
- Large sessions in batch jobs: Use session.clear() periodically or StatelessSession
- Missing DB indexes: Add indexes on Hibernate query filter columns
- No readOnly hint: Use @Transactional(readOnly=true) for read-only operations
- Full entity loads for reporting: Use projections/DTOs to fetch only needed columns
- No batch inserts: Set hibernate.jdbc.batch_size=50 and order_inserts=true
- Open Session in View: Hides real design problems; disable in REST APIs

★ Q20: How does Hibernate handle schema validation and migrations alongside Flyway or Liquibase? [FREQUENTLY ASKED]

In production, schema management belongs to dedicated migration tools, not Hibernate DDL auto.

- Flyway: SQL migration files (V1__init.sql, V2__add_column.sql). Runs on startup, tracks history in flyway_schema_history.
- Liquibase: XML/YAML/SQL changelogs with rollback support and environment profiles.

```
spring.jpa.hibernate.ddl-auto=validate
```

```
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
```

Workflow: Flyway runs first (creates/alters tables), then Hibernate validates entity mappings match. If mismatch, startup fails — preventing schema drift reaching production.

Best Practice: Never use ddl-auto=update in production — it won't drop columns and makes schema changes invisible to version control.

★ Q21: How do you enable and interpret Hibernate's SQL logging for debugging? [FREQUENTLY ASKED]

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

What to look for:

- Multiple SELECTs for same entity type in one request = N+1 problem
- UPDATE fired on every request even with no changes = unintended dirty checking
- No WHERE clause = missing index or missing filter
- Many duplicate rows = missing DISTINCT on JOIN FETCH with collections

Advanced tools: datasource-proxy (count queries per request), P6Spy (SQL with timing), Hibernate Statistics (enable via hibernate.generate_statistics=true).

★ Q22: What is the difference between session.save(), session.persist(), session.merge(), session.update(), and session.saveOrUpdate()? [FREQUENTLY ASKED]

Method	Behavior	Notes
save()	Inserts entity, returns generated ID immediately	Hibernate-specific; may fire INSERT eagerly
persist()	JPA standard; schedules INSERT, no return value	INSERT fires on flush, not immediately
merge()	Copies state to managed entity, returns managed copy	Original stays detached; use the returned copy
update()	Re-attaches detached entity to session	Throws if same ID already in session
saveOrUpdate()	Inserts if transient, updates if detached	Hibernate convenience method

Section 2: Spring Data JPA

This section covers Spring Data JPA from fundamentals to advanced topics including repositories, queries, relationships, transactions, and testing. Questions marked with ★ are most frequently asked.

Part A: Core Concepts & Fundamentals

★ Q1: What is Spring Data JPA and how does it differ from plain JPA? [FREQUENTLY ASKED]

Spring Data JPA is a Spring module that auto-generates repository implementations at runtime, eliminating boilerplate persistence code.

- Plain JPA: Manual EntityManager, hand-written JPQL, transaction management, pagination logic
- Spring Data JPA: Extend JpaRepository, declare method signatures — Spring generates all code

```
public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByEmailContainingAndActiveTrue(String email);
    Optional<User> findFirstByOrderByCreatedAtDesc();
    // No implementation needed!
}
```

★ Q2: What is the role of JpaRepository and what methods does it provide? [FREQUENTLY ASKED]

JpaRepository extends PagingAndSortingRepository which extends CrudRepository. It provides:

- save(S entity) / saveAll(Iterable) — insert or update
- findById(ID) — returns Optional<T>
- findAll() / findAll(Sort) / findAll(Pageable) — retrieve all with paging/sorting
- existsById(ID) — check existence without loading entity
- deleteById(ID) / delete(T) / deleteAll()
- count() — total record count
- flush() / saveAndFlush() — force immediate persistence to DB
- getReferenceById(ID) — returns proxy without DB hit
- findAllById(Iterable<ID>) — batch find by IDs

★ Q3: What is the difference between CrudRepository, JpaRepository, and PagingAndSortingRepository? [FREQUENTLY ASKED]

- CrudRepository<T,ID>: Base with save, findById, findAll, delete, count. Framework-agnostic — works with MongoDB, Redis, JPA.
- PagingAndSortingRepository<T,ID>: Adds findAll(Sort) and findAll(Pageable) for pagination and sorting.
- JpaRepository<T,ID>: Adds JPA-specific: flush(), saveAndFlush(), deleteInBatch(), getReferenceById(), findAll(Example).

Hierarchy: CrudRepository < PagingAndSortingRepository < JpaRepository
Use JpaRepository in most Spring Boot + JPA projects.

★ Q4: How does Spring Data JPA auto-generate query implementations at runtime? [FREQUENTLY ASKED]

- At startup, Spring scans interfaces extending Repository<T,ID>
- For each method, inspects method name or @Query annotation
- Derived methods parsed by PartTreeJpaQuery: tokenized into Subject (find/count/exists) + Predicates (ByField, AndField)
- @Query methods compiled to NamedQuery or executed as-is
- JpaRepositoryFactory creates a JDK dynamic proxy, binding each method to its query strategy

```
// 'findByLastNameAndAgeGreaterThan' becomes:
// SELECT u FROM User u WHERE u.lastName = ?1 AND u.age > ?2
```

★ Q5: What is a derived query method? Give an example. [FREQUENTLY ASKED]

A derived query method is one whose query is auto-generated by Spring Data JPA from the method name following a naming convention.

```
public interface UserRepository extends JpaRepository<User, Long> {
    // WHERE u.email = ?1
    Optional<User> findByEmail(String email);

    // WHERE u.age > ?1 AND u.active = true
    List<User> findByAgeGreaterThanAndActiveTrue(int age);

    // COUNT WHERE u.department = ?1
    long countByDepartment(String department);

    // WHERE u.name LIKE %?1%
    List<User> findByNameContaining(String name);
}
```

Subject keywords: findBy, countBy, existsBy, deleteBy. Predicate keywords: And, Or, Between, LessThan, GreaterThan, Like, Containing, In, OrderBy, True, False.

★ Q6: How does the @Query annotation work and when would you use it over derived queries? [FREQUENTLY ASKED]

@Query lets you define JPQL or native SQL directly, bypassing the naming convention parser.

```
// JPQL
@Query('SELECT u FROM User u WHERE u.email = :email AND u.active = true')
Optional<User> findActiveByEmail(@Param('email') String email);

// Native SQL
@Query(value='SELECT * FROM users WHERE email = ?1', nativeQuery=true)
Optional<User> findByEmailNative(String email);
```

Use @Query when:

- Derived method name would be too long or unreadable
- Query involves JOINs, GROUP BY, HAVING, or aggregations
- Native SQL features needed (DB-specific functions, UNION)
- Complex query structure for performance optimization

★ Q7: What are the different ID generation strategies? [FREQUENTLY ASKED]

- AUTO: JPA picks based on database (SEQUENCE for PostgreSQL/Oracle, IDENTITY for MySQL)
- IDENTITY: Uses DB auto-increment. ID known only after INSERT — breaks batch inserts.
- SEQUENCE: Uses a DB sequence object. Can pre-allocate IDs. Best for batch inserts.
- TABLE: Simulates sequences with a table. Portable but has locking overhead — rarely used.

```
@Id
@GeneratedValue(strategy=GenerationType.SEQUENCE, generator='user_seq')
@SequenceGenerator(name='user_seq', sequenceName='user_sequence',
                   allocationSize=50)
private Long id;
```

Best Practice: Use SEQUENCE with allocationSize > 1 for batch insert performance. IDENTITY forces a DB round-trip per insert.

★ Q8: What is the difference between save() and saveAndFlush() in JpaRepository? [FREQUENTLY ASKED]

- save(entity): Persists or merges entity into the persistence context. SQL INSERT/UPDATE may be deferred until flush (end of transaction or explicit flush call).
- saveAndFlush(entity): Persists and immediately forces a flush. SQL executes synchronously before the method returns.

```
// save() -- SQL deferred, may be batched
User user = userRepo.save(new User('John'));

// saveAndFlush() -- SQL fires immediately
User user = userRepo.saveAndFlush(new User('John'));
```

Use saveAndFlush() when a subsequent native query in the same transaction needs the persisted data to be visible.

★ Q9: What is the difference between EAGER and LAZY loading? Which is the default for each relationship type? [FREQUENTLY ASKED]

- LAZY: Association loaded only when accessed. Proxy/placeholder returned initially. Default for @OneToMany and @ManyToMany.
- EAGER: Association loaded immediately with the parent entity. Default for @ManyToOne and @OneToOne.

Annotation	Default Fetch	Recommended
@OneToMany	LAZY	LAZY (keep)

@ManyToMany	LAZY	LAZY (keep)
@ManyToOne	EAGER	LAZY (change)
@OneToOne	EAGER	LAZY (consider)

Best Practice: Keep all relationships LAZY and explicitly load via @EntityGraph or JOIN FETCH. EAGER loading causes hidden performance issues.

★ Q10: What is the N+1 select problem and how do you solve it in Spring Data JPA? [FREQUENTLY ASKED]

Loading a list triggers one query, then N more queries for each entity's lazy association.

Solutions:

- @EntityGraph on repository method (most idiomatic):
```
@EntityGraph(attributePaths = {"orders", "orders.items"})
List<Customer> findAllByActiveTrue();
```
- @Query with JOIN FETCH:
```
@Query('SELECT DISTINCT c FROM Customer c JOIN FETCH c.orders')
List<Customer> findAllWithOrders();
```
- @BatchSize on collection: @BatchSize(size = 25) — loads in batches
- Use DTOs/Projections to fetch flat data without associations

Part B: Relationships, Queries & Advanced Topics

★ Q11: How do you use @EntityGraph to avoid N+1 queries? [FREQUENTLY ASKED]

@EntityGraph tells Spring Data to eagerly fetch specified associations for a specific repository method via a JOIN, without changing the entity's global FetchType.

```
// Ad-hoc @EntityGraph
@EntityGraph(attributePaths = {"department", "skills"})
List<Employee> findByStatus(String status);
```

Named EntityGraph (on entity):

```
@Entity
@NamedEntityGraph(name='Employee.detail', attributeNodes = {
    @NamedAttributeNode('department'),
    @NamedAttributeNode(value='projects', subgraph='projects')
})
public class Employee { ... }

@EntityGraph('Employee.detail')
List<Employee> findAll();
```

★ Q12: What is a Specification in Spring Data JPA and when should you use it? [FREQUENTLY ASKED]

Specification is a Spring Data abstraction over JPA Criteria API for composable, reusable query predicates.

```
public class UserSpec {
```

```
    public static Specification<User> hasEmail(String email) {
        return (root, query, cb) -> cb.equal(root.get('email'), email);
    }
    public static Specification<User> isActive() {
        return (root, query, cb) -> cb.isTrue(root.get('active'));
    }
}

// Composable:
userRepository.findAll(UserSpec.hasEmail(email).and(UserSpec.isActive()));
```

Requires repository to extend JpaSpecificationExecutor<T>. Use for dynamic search forms with varying filter criteria at runtime.

★ Q13: What is a Projection and how is it used to fetch partial data? [FREQUENTLY ASKED]

Projections fetch a subset of entity fields, reducing data transfer and memory usage.

Interface-based projection:

```
public interface UserSummary {
    String getName(); String getEmail();
}
List<UserSummary> findByDepartment(String dept);
// Generates: SELECT u.name, u.email FROM users WHERE u.department = ?
```

Class-based (DTO) projection:

```
public record UserDTO(String name, String email) {}

@Query('SELECT new com.example.UserDTO(u.name, u.email) FROM User u
       WHERE u.department = :dept')
List<UserDTO> findDtoByDepartment(String dept);
```

Closed projections (only declared getters) generate optimized SELECT with only needed columns. Open projections (@Value SpEL) load the full entity.

★ Q14: How do you implement pagination and sorting using Pageable? [FREQUENTLY ASKED]

```
// Repository
Page<User> findByDepartment(String dept, Pageable pageable);

// Service
Pageable page = PageRequest.of(0, 20,
    Sort.by('lastName').ascending().and(Sort.by('firstName')));
Page<User> result = userRepo.findByDepartment('Engineering', page);

// Metadata
result.getTotalElements(); // total matching records
result.getTotalPages();
result.getContent();       // List<User> for this page
result.hasNext();
```

For large datasets where COUNT(*) is expensive, use Slice<T> — it avoids the count query and only checks if there is a next page.

★ Q15: What is the use of @Modifying with @Query and what precautions should be taken? [FREQUENTLY ASKED]

@Modifying signals that a @Query method performs UPDATE or DELETE, not a SELECT.

```
@Modifying
@Transactional
@Query('UPDATE User u SET u.active = false WHERE u.lastLogin < :date')
int deactivateInactiveUsers(@Param('date') LocalDate date);
```

Precautions:

- Always add @Transactional — @Modifying without a transaction throws
- Bypasses persistence context — in-memory entities are NOT updated, only DB rows
- Add clearAutomatically=true to evict stale entities from L1 cache
- flushAutomatically=true flushes pending changes before the bulk update
- Returns int (affected rows) or void

★ Q16: How does Spring Data JPA handle optimistic locking with @Version? [FREQUENTLY ASKED]

```
@Entity public class Product {
    @Id Long id;
    String name;
    @Version Long version; // Hibernate auto-manages
}
```

- On UPDATE: WHERE id = ? AND version = ? — fails if version changed by another transaction
- Hibernate auto-increments version after successful update
- Conflict throws OptimisticLockException (JPA) or StaleObjectStateException (Hibernate)

```
try { productRepo.save(product); }
catch (OptimisticLockingFailureException e) {
    // Re-read latest and retry, or notify user of conflict
}
```

★ Q17: How do you handle cascade types — PERSIST, MERGE, REMOVE, ALL, DETACH, REFRESH? [FREQUENTLY ASKED]

- PERSIST: Saving parent also saves new child entities
- MERGE: Merging parent also merges associated child changes
- REMOVE: Deleting parent also deletes all children — use with caution!
- DETACH: Detaching parent also detaches children
- REFRESH: Reloading parent from DB also refreshes children
- ALL: All of the above — convenient but dangerous with REMOVE

```
@OneToMany(mappedBy='order',
```

```
            cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    private List<OrderItem> items; // REMOVE intentionally excluded
```

Best Practice: Avoid CascadeType.REMOVE on large collections — it loads all children into memory to delete them one by one.

★ Q18: What is the purpose of @Embedded and @Embeddable? [FREQUENTLY ASKED]

@Embeddable marks a class whose fields are stored in the owning entity's table (no separate table). @Embedded is used on the field in the owner.

```
@Embeddable public class Address {
    private String street, city, pincode;
}

@Entity public class Customer {
    @Id Long id; String name;
    @Embedded Address shippingAddress;
    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name='street',
                           column=@Column(name='bill_street'))
    })
    Address billingAddress;
}
```

All Address fields are in the Customer table. Use @AttributeOverride to rename columns when embedding the same class twice.

★ Q19: What is the use of QueryDSL with Spring Data JPA? [FREQUENTLY ASKED]

QueryDSL provides compile-time type-safe queries with generated Q-classes, as a more readable alternative to Criteria API.

```
// Type-safe predicate
QUser user = QUser.user;
Predicate pred = user.age.gt(18)
    .and(user.email.endsWith('@company.com'));

userRepository.findAll(pred, PageRequest.of(0, 20));
```

- Requires: querydsl-jpa + querydsl-apt (generates Q-classes from @Entity)
- Repository extends QuerydslPredicateExecutor<T>
- QueryDSL vs Specification: QueryDSL is more readable/type-safe; Specification is built-in with less setup

★ Q20: How would you test a Spring Data JPA repository using @DataJpaTest? [FREQUENTLY ASKED]

@DataJpaTest is a slice test that starts only the JPA layer with an in-memory H2 database. Each test rolls back after completion.

```
@DataJpaTest
class UserRepositoryTest {
```

```
    @Autowired private UserRepository userRepo;
    @Autowired private TestEntityManager em;

    @Test
    void findByEmail_shouldReturnUser() {
        User user = em.persistAndFlush(new User('test@test.com'));
        Optional<User> found = userRepo.findByEmail('test@test.com');
        assertThat(found).isPresent();
    }
}
```

TestEntityManager lets you set up test data and flush to make it queryable. Each test runs in a transaction that auto-rolls back.

★ Q21: What is the difference between @OneToMany(mappedBy=...) and @JoinColumn? [FREQUENTLY ASKED]

- @JoinColumn on @ManyToOne / @OneToOne: This entity owns the FK column in its table.
- mappedBy on @OneToMany: Inverse (non-owning) side. FK managed by the named field on the child entity. No FK column in the parent table.

```
@Entity public class Order {
    @OneToMany(mappedBy='order') // 'order' = field in OrderItem
    List<OrderItem> items;
}
@Entity public class OrderItem {
    @ManyToOne
    @JoinColumn(name='order_id') // FK column lives here
    Order order;
}
```

Common mistake: Setting both mappedBy AND @JoinColumn on the same relationship side causes Hibernate to generate incorrect SQL with duplicate FK columns.

★ Q22: What is the difference between findById() returning Optional vs getReferenceById() returning a proxy? [FREQUENTLY ASKED]

- findById(ID): Fires SELECT immediately. Returns Optional<T> — empty if not found. Use when you need to read entity data.
- getReferenceById(ID): Returns a proxy without hitting DB. Throws EntityNotFoundException at first field access if entity doesn't exist. Use to set FK references.

```
// findById — need to read data
User user = userRepo.findById(1L)
    .orElseThrow(() -> new ResourceNotFoundException('Not found'));

// getReferenceById — just setting a FK, no data read needed
Order order = new Order();
order.setUser(userRepo.getReferenceById(userId)); // no DB hit
orderRepo.save(order);
```

★ Q23: What is the use of @Lock annotation and what lock types are available? [FREQUENTLY ASKED]

@Lock on a repository method sets the JPA LockModeType for that query.

LockModeType	Behavior
PESSIMISTIC_READ	Shared lock — others can read, not write
PESSIMISTIC_WRITE	Exclusive lock — SELECT FOR UPDATE
PESSIMISTIC_FORCE_INCREMENT	Exclusive + increments @Version
OPTIMISTIC	Validates version on commit (default for @Version entities)
OPTIMISTIC_FORCE_INCREMENT	Reads and increments @Version even for reads
NONE	No locking (default)

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query('SELECT a FROM Account a WHERE a.id = :id')
Account findByIdForUpdate(@Param('id') Long id);
```

★ Q24: Explain auditing in Spring Data JPA using @CreatedDate, @LastModifiedDate, @EnableJpaAuditing. [FREQUENTLY ASKED]

```
@SpringBootApplication @EnableJpaAuditing
public class Application { ... }

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class Auditable {
    @CreatedDate @Column(updatable=false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy @Column(updatable=false)
    private String createdBy;
}
```

For @CreatedBy / @LastModifiedBy, implement AuditorAware<String> bean returning the current authenticated user from Spring Security context.

★ Q25: What is the purpose of orphanRemoval = true in relationship mappings? [FREQUENTLY ASKED]

orphanRemoval=true automatically deletes child entities when removed from their parent's collection, even without explicitly calling delete().

```
@OneToMany(mappedBy='cart', cascade=CascadeType.ALL, orphanRemoval=true)
private List<CartItem> items;

cart.getItems().remove(0); // triggers DELETE on removed CartItem
```

```
cartRepo.save(cart);
```

- Different from CascadeType.REMOVE: REMOVE deletes children when parent is deleted; orphanRemoval deletes children when they lose their parent reference
- Use only on @OneToMany/@OneToOne where child has no meaning without parent
- Never use on @ManyToMany — would incorrectly delete shared entities

★ Q26: What are common performance tuning strategies when using Spring Data JPA? [FREQUENTLY ASKED]

- Keep all FetchType LAZY; eagerly load per-query with @EntityGraph
- Use @Transactional(readOnly=true) on all read service methods
- Set hibernate.jdbc.batch_size=50 and order_inserts=true for batch saves
- Use Pageable — never findAll() on large tables
- Monitor SQL with show_sql=true or datasource-proxy in development
- Use @Modifying bulk queries instead of load-then-delete patterns
- Use DTOs/Projections for reporting — avoid loading full entity graphs
- Add DB indexes on columns used in findBy methods and JOIN conditions
- Cache frequently read, rarely changing data with @Cacheable
- Use @DynamicUpdate on wide entities to update only changed columns

Quick Reference Cheat Sheet

Hibernate Entity States

State	Condition	Tracked?
Transient	new Entity() — no Session, no DB row	No
Persistent	Linked to open Session with DB row	Yes — dirty checked
Detached	Session closed — has ID but not tracked	No — must merge()
Removed	session.delete() called	Yes — deleted on flush

Fetch Type Defaults & Recommendations

Annotation	Default	Recommended
@OneToMany	LAZY	LAZY (keep)
@ManyToMany	LAZY	LAZY (keep)
@ManyToOne	EAGER	LAZY (change it!)
@OneToOne	EAGER	LAZY (consider)

DDL Auto Strategy Guide

Strategy	Behavior	Use In
none	No schema action	Production
validate	Validates schema vs entities; fails on mismatch	Production
update	Adds missing; never drops columns	Dev only
create	Drops + recreates on startup	Dev / Testing
create-drop	Creates on start; drops on stop	Integration tests

Repository Hierarchy

```
Repository<T, ID>
  └── CrudRepository<T, ID>
        (save, findById, findAll, delete, count)
        └── PagingAndSortingRepository<T, ID>
              (+findAll(Pageable), findAll(Sort))
              └── JpaRepository<T, ID>
                    (+flush, saveAndFlush, getReferenceById)
```

N+1 Solutions Quick Reference

Solution	When to Use
JOIN FETCH in HQL	Single method needing joined data
@EntityGraph	Specific repository method — most idiomatic
@BatchSize(size=N)	When JOIN FETCH causes cartesian product issues
DTO Projection	Read-only operations, reporting, display-only
@Fetch(SUBSELECT)	All collections loaded with one subquery

Key Annotation Reference

Annotation	Purpose
@Entity	Marks a class as a JPA entity mapped to a DB table
@Table(name=...)	Specifies the DB table name
@Id	Marks the primary key field
@GeneratedValue	Configures ID generation strategy
@Version	Enables optimistic locking; auto-incremented on update
@Transient	Field not persisted to DB
@Embedded / @Embeddable	Embeds value object fields into owner table
@OneToMany(mappedBy)	Inverse side of one-to-many; no FK column here
@ManyToOne @JoinColumn	Owns the FK column in this entity's table
@EntityGraph	Per-method eager loading without changing FetchType
@Modifying	Required for UPDATE/DELETE @Query methods
@Cache / @Cacheable	Enables second-level caching for an entity
@DynamicUpdate	Only UPDATE columns that actually changed
@BatchSize(size=N)	Load associations in batches, not one-by-one

Performance Best Practices

- Keep FetchType=LAZY on all relationships; eagerly load per-query with @EntityGraph
- Use @Transactional(readOnly=true) on all read-only service methods
- Set hibernate.jdbc.batch_size=50 and order_inserts=true for bulk operations
- Use Pageable — never call findAll() on large tables
- Monitor SQL with show_sql=true and datasource-proxy in development
- Use ddl-auto=validate in production; manage migrations with Flyway or Liquibase

- Use @DynamicUpdate on wide entities to avoid updating unchanged fields
- Use DTOs/Projections for reporting — avoid loading full entity graphs
- Use @Modifying bulk queries instead of load-then-delete patterns in memory